

Traiter les questions suivantes :**Exercice 1: (11 pts)**

On considère la classe *Car* suivante:

```
public class Car {                // voiture en français
    private String brand;         // marque en français
    private int year;             // année en français
    private String country;       // pays en français

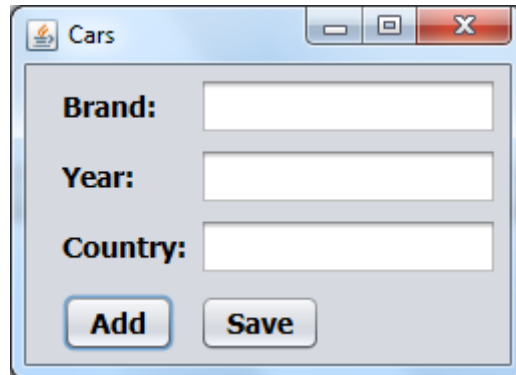
    public Car(String brand, int year, String country) {
        this.brand = brand;
        this.year = year;
        this.country = country;
    }

    public int getYear() {
        return year;
    }

    public String getCountry() {
        return country;
    }

    public String getBrand() {
        return brand;
    }
}
```

On considère l'interface graphique suivante permettant de saisir les données d'une voiture dans des champs de texte. Le bouton *Add* permet de créer un nouvel objet *Car* et de l'ajouter à un *ArrayList*. Le bouton *Save* permet de sauvegarder les voitures de l'*ArrayList* dans un fichier. Les données du fichier sont chargées au démarrage du programme à l'aide de la méthode *load* ci-dessous.



Le code correspondant à cette interface graphique est comme suit:

```
public class CarFrame extends Frame {
    private TextField brand = new TextField(10);
    private TextField year = new TextField(10);
    private TextField country = new TextField(10);
    private Label lBrand = new Label("Brand");
    private Label lYear = new Label("Year");
    private Label lCountry = new Label("Country");
    private Button add = new Button("Add");
    private Button save = new Button("Save");
    private ArrayList<Car> carsList = new ArrayList<Car>();

    public CarFrame() {
        // à compléter ... (1)
        this.pack();
        load();
        // à compléter ... (2)
    }

    private void addPerformed() {
        // à compléter ... (3)
    }

    private void savePerformed() {
        // à compléter ... (4)
    }
}
```

```
public void save2Performed() {
    // à compléter ... (5)
}

private void load() {
    // à compléter ... (6)
}
}
```

- Donner en (1) le code nécessaire afin d'obtenir l'interface graphique ci-dessus. (Vous pouvez utiliser les gestionnaires *BorderLayout*, *FlowLayout* et *GridLayout*).
- Donner en (2) le code qui permet d'ajouter un écouteur d'action (*ActionListener*) au bouton *Add* et qui exécute la méthode *addPerformed*.
- Donner en (3) le code permettant de saisir les champs de texte, de créer un objet *Car* et de l'ajouter à l'objet *carsList*. Les champs doivent être vides après l'ajout.
- Donner en (4) le code permettant d'ouvrir le fichier « car.txt » et d'y écrire le contenu de la *carsList*. Chaque ligne doit contenir les informations d'une seule voiture comme suit :
Mercedes, 1990, Germany
- Donner en (5) le code permettant de sérialiser (persister) les objets *Car* de la *carsList* dans un fichier « data.bin ». Quel changement doit-on effectuer à la classe *Car* pour la persistance?
- Donner en (6) le code permettant de charger les informations contenues dans le fichier « car.txt », de créer des objets *Car* (un par ligne) et de les ajouter à la *carsList*.
- La méthode *parseInt* soulève une exception du type *NumberFormatException* si la String passée en paramètre ne peut pas être convertie en un nombre entier. Changer le code de la méthode *addPerformed()* afin de supporter cette exception. L'appel de cette exception affichera un *JOptionPane* comme suit:



Exercice 2: (5 pts)

- Ecrire le code permettant de dessiner un rectangle avec la couleur verte et de le remplir avec la couleur rouge. Les dimensions du rectangle sont entrées dans le code html.
- Modifier le code précédant pour afficher un message d'erreur si les paramètres du rectangle sont supérieurs aux dimensions de l'applet.

Exercice 3: (4 pts)

Supposons qu'on a une base de données appelée « customerDB » avec un username « tripoli » et un password « saida ».

Cette base de données contient la table « Customer » qui est composée de deux attributs: « id » et « name » avec les tuples (enregistrements) suivants:

id	name
10	Sky Net
20	Galaxy
30	Bon Jus
40	Pro Cars

- Ouvrir une connexion à la base de données, lire les données de cette table et les placer dans un objet de type *HashMap*.
- Utiliser un itérateur sur la *HashMap* déjà créée et ensuite afficher chaque tuple sur une ligne.